

Novel Usage of Machine Learning and Meta-heuristics in Stochastic Resonance Pre-emphasis Filtering

Noah Marosok
Mathieu Antonopoulos

Contents

1	Introduction	2
1.1	Related Work	2
1.2	Problem Formulation	2
2	Methods	3
3	Results	5
4	Conclusions	12

List of Tables

1	Comparison of Previous literature with the methods in discussion	7
---	--	---

List of Figures

1	Peak to Peak calculation of SR output	4
2	Peak to RMS calculation of SR output	4
3	5 second signal slice	4
4	Slice of an intracellular signal from 100 to 105 seconds. All SR output waveforms are translated and scaled to show a visually comparable analysis	6
5	Signal with additive GWN	8
6	Result of min batch K means clustering on the SR filtered output	9
7	Power Scaled Noise on top of Filtered Noisy Signal	9
8	Correctly Power Scaled Noise Profile overlay with filtered signal output	10
9	One Class SVM output on Filtered Signal with 8 as noise profile	11
10	Raw signal slice	11

1 Introduction

With the emergence of newer bio-integrateable computational electronics comes the problems associated with the unknown aspects and processes of the human body. In an ideal scenario, electrodes can accurately determine the process or response the brain is trying to mimic by measuring the electrical impulse of the spike trains. As one would assume, ideal cases do not exist in reality. Therefore it is paramount to be able to find ways to accurately measure these signals in a dynamic manner, that changes and adjusts according to the subject being studied. Additive noise in the signal can be detrimental to decoding if certain neurons are being measured are swamped by spikes from nearby neurons. Also, with the introduction of foreign bodies into the brain leads to a process called gliosis. Gliosis is a defense mechanism within the human body that encapsulates foreign bodies within special tissue called glial scar tissue. This scar tissue adds impedance to the neural signals and can therefore decrease the accuracy of neural measurement. In addition to these scenarios, live subjects move around their environment, which will inevitably shift the electrode within the subjects brain, which leads to change in background noise levels and potential change in location from the point of interest. To address these concerns, filtering methods have been thoroughly tested and compared to most accurately determine the proper encoding of these signals.

1.1 Related Work

Stochastic resonance is a method that utilizes the power of the noise of an input signal in order to filter more effectively. This is done by imagining a brownian particle inside of a potential well [2],[1], U_x , where the noisy signal is applied as a velocity vector onto the Brownian particle and $-U'_x$ applied as a vector in the opposite direction, tangential to any point along U_x . The displacement along the x axis produces the output signal for any data point $x[n]$. Stochastic resonance has been shown to outperform advanced methods such as bandpass filtering, wavelet transform, and Teager Energy Operator. These methods will not be discussed in detail in this literature. Thus far, the work in SR pre-emphasis primarily consists of investigation into the efficacy and parameter selection of the SR potential well, that is the values of the equation that formulate the shape and hence the slope of U_x . The way in which this work differs from previous literature, is the iterative nature, and pragmatism of the iterative nature of this method. Previously, solution spaces for the well parameters were simulated for sweeping values of the well parameters, then hand selected as the optimal solution candidate. In addition to hand selecting values within the solution space, ground truth data from the input signal is known. That is, spike portions and noise portions of the signal are clearly known ahead of SR, which leads to an overconfident filtering performance. In this work I seek to not only optimize the solution to the differential to maximize SR output SNR, but also to do so in a method that is completely blind of any ground truth data in the signal and can accurately determine spikes within the signal with high confidence.

1.2 Problem Formulation

Based off of the previous description of the work, the overall goal is to maximize the output SNR of the signal in a manner that can be reproduced on blind signals in any neuro lab around the world. For this work specifically, this will be tested and implemented with a 750 second long neural signal taken at 20 kHz sampling rate which is an intracellular recording from the hippocampus region CA1 of rats from the Collaborative Research in Computational Neuroscience database. This signal contains 628 spikes within and due to its nature as an intracellular signal, has little noise at all. Due to this, synthetic additive noise is created via zero mean Gaussian white noise at 1 W. The given

constraints of this project are that the algorithm must be somewhat computationally expedient, it must have generalizability to be able to be useful to the field of research, and it must be able to be dynamic without any advanced user input.

2 Methods

In the goal of optimization, a cost efficient global optimization algorithm must be used to find not necessarily the optimal solution, but a suitable solution that provides an increase in ΔSNR performance. The heuristic method used to optimize the solution is the Particle Swarm Optimization algorithm. This algorithm is a nature inspired algorithm that seeks to find a global optimum based off of a phenomenon in nature known as 'swarm intelligence.' This correctly implies that the searching method of each individual particle in the swarm is also determinant upon the social cohesion of the swarm. A particle in the swarm is defined by a position vector $X^i(t) = (x^i(t), y^i(t))$ with the update rule

$$X^i(t+1) = X^i(t) + V^i(t+1)$$

where

$$V^i(t+1) = wV^i(t) + c_1r_1(pbest^i - X^i(t)) + c_2r_2(gbest - X^i(t))$$

with w as the inertia coefficient, c_1 and c_2 as the cognitive and social coefficient respectively, $pbest$ and $gbest$ as the i th particle best and global particle best respectively, and r_1 and r_2 as random integers $\in [0, 1]$. Thus, the PSO is as follows:

1. Initialize n particles within the bounds of the search space
2. Do for each particle i :
 - (a) calculate the next position based off of velocity term
 - (b) if next position is better than current position then update the particle position
3. Calculate the global best particle position
4. Repeat steps 2-3 until a desirable solution is found

In order to determine a proper optimization objective function, the two methods proposed, peak to RMS and peak to peak, will be compared. The objective function for peak to peak is

$$f_{pp}(s_n) = \max(s_n) - \min(s_n) \tag{1}$$

For the peak to RMS method, the objective function is

$$f_{pRMS}(s'_n) = \max(s'_n) - \text{RMS}_{s'_n} \tag{2}$$

where s'_n is the absolute value of the SR output. Visually,

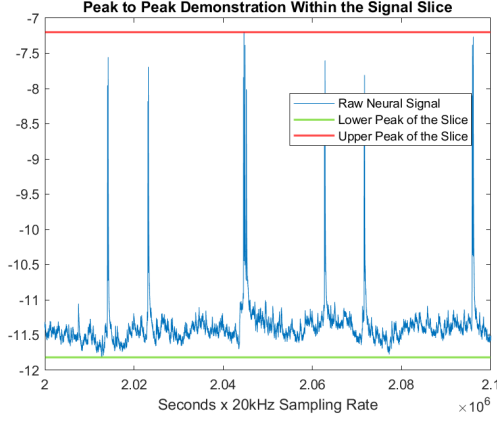


Figure 1: Peak to Peak calculation of SR output

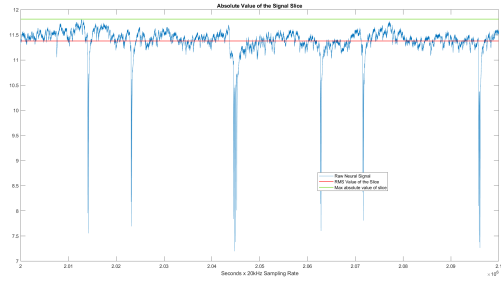


Figure 2: Peak to RMS calculation of SR output

We then utilize both of these objective functions as input into the particle swarm optimization algorithm which seeks to find the global minimum of an objective function. Since the desired effect is a global maximum of the objective function, we simply input the objective functions multiplied by negative one. These two blind objective functions will be compared with the non-optimized, non-blind solution and with the optimized, non-blind solution to validate their performance.

To be able to visually compare the effects of the proposed methods, a 5 second slice of each signal will be presented. The raw signal slice

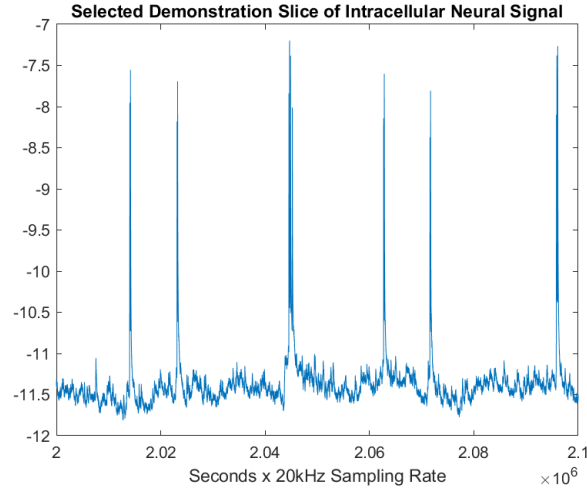


Figure 3: 5 second signal slice

runs from 100 to 105 seconds and contains six spike portions with noticeable noise portions before the additive noise.

After determining the ideal optimization objective function that produces the best output, we can

then implement a one class SVM to label spike and noise portions in the signal. A one class SVM works almost identically to an SVM except instead of using the traditionally conceptualized hyperplane decision boundary with a maximized margin, it instead uses a decision hypersphere. After training the SVM utilizing the radial basis function (RBF) kernel, it produces support vectors that can make predictions upon our signal. The output of the SVM produces decision values that can be either negative or positive. Negative values are classified as outliers while expected values are positive. Thresholding within the decision values can determine the degree of confidence which we wish our model to exhibit. Due to previous experimentation upon the data set, it was decided that thresholding set at 0 was sufficient for the classification performance, with additional post processing upon the SVM output. The SVM is trained with a SR output (with the same parameters as the signal SR output) of a Gaussian white noise profile that had an original power of 1 W. To more accurately model a relationship between the two signals, a scaling factor between the SR output of the neural signal is used where

$$\text{ScalingFactor} = \frac{P_{\text{signal}}}{P_{\text{noise}}} \quad (3)$$

which is then multiplied to the SR output of the noise profile. This should give a noise profile that has the same magnitude as the noise of the signal.

As an alternative to one class SVM, a variant of the k-means algorithm will be used. Min-batch k-means is an algorithm that is suitable for large datasets which instead clusters the signal in batches. This is useful because it is parallelizable

3 Results

In the optimization implementations, using the same random seed for reproducibility, it was found that the optimized solution utilizing the non-blind SNR calculation as an objective function produced better results than the previous method. The other two methods being tested, intuitively, one could assume that maximizing these two values will also lead to inherent SNR improvement in the SR output. As 1 shows, the peak to peak optimization showed declined performance in total SNR performance from the non-optimized, non-blind, solution, but drastic reduction in SNR performance to the non-blind, optimized solution. The RMS to peak optimization also showed extremely poor performance for the given iteration.

Investigating these selected values, one would assume that given the poor performance of the peak to SNR optimization, that it should be discarded as a method. However, upon visual comparison of the selected optimization methods 4, it shows that peak to RMS shows a more characteristic spike profile than the supposedly higher performing peak to peak method. Given the optimization method is completely blind, it follows that it should not perform as well as a non-blind optimization, which is evident visually 4.

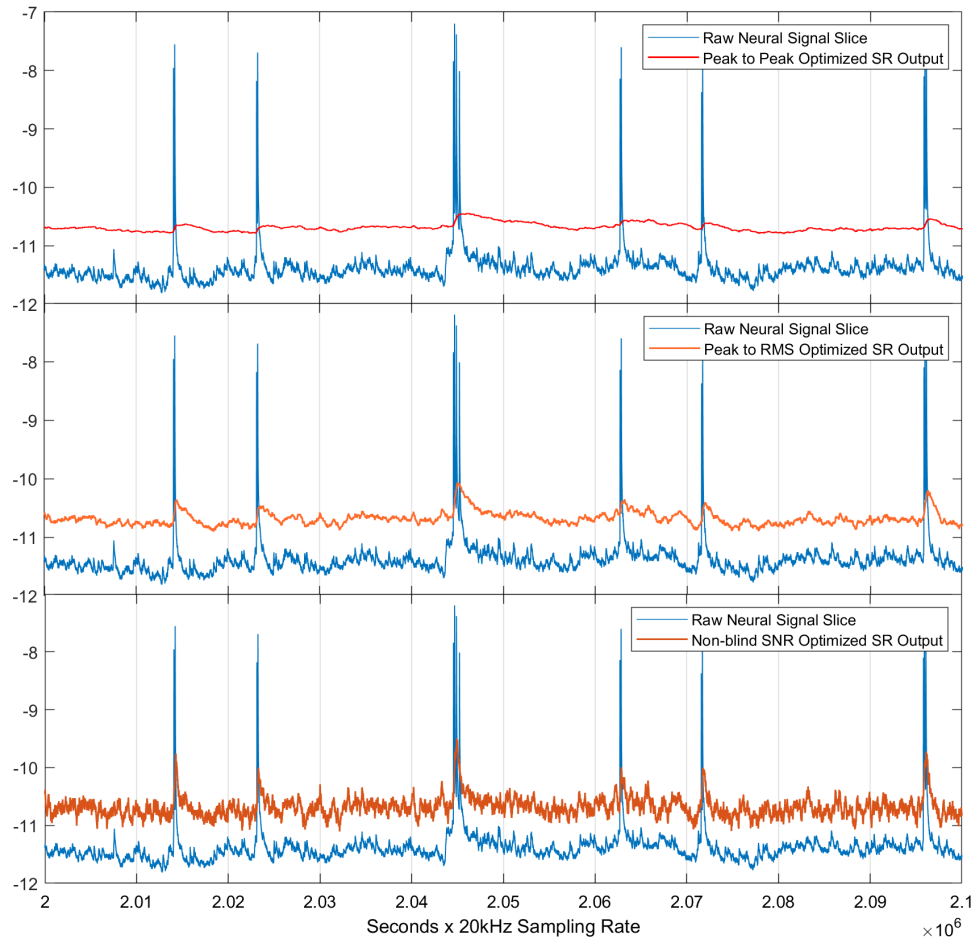


Figure 4: Slice of an intracellular signal from 100 to 105 seconds. All SR output waveforms are translated and scaled to show a visually comparable analysis

Parameter Selection through Optimization	Non-Blind SNR Calculation		Blind SNR Calculation	
	C. Gungor et al.	Optimized Solution	RMS to Peak Optimization	Peak to Peak Optimization
a	1000	1079	823.65	1189
h	1.471e-5	7.985e-6	5.0369e-6	7.432e-5
SNR	1.2409	1.549	-1.5706	0.1993

Table 1: Comparison of Previous literature with the methods in discussion

Running this optimization 20 times to find the overall reliability of these methods, it was found that the non-blind optimization had a mean SNR of 1.342 with a standard deviation of 0.5324. The peak to RMS and peak to peak methods produced means of -1.862 and -1.169 with standard deviations of 1.1354 and 0.5186, respectively.

After deciding upon the RMS to peak optimization as the more visually apparent spike performance for our specific signal slice, we utilize the parameter values in 1 for all SR filtering that is done henceforth. Prior to our signal being filtered, we are left with

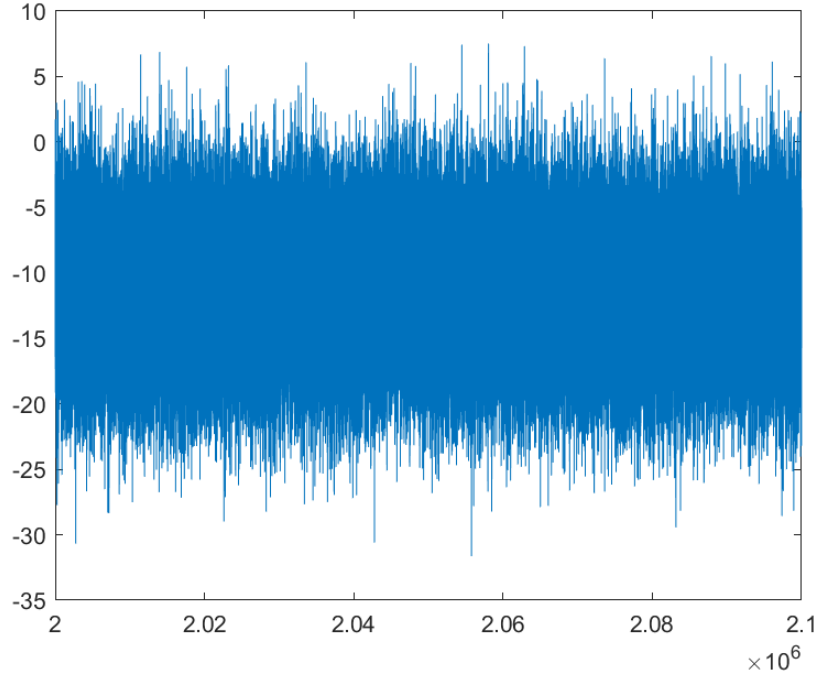


Figure 5: Signal with additive GWN

Initially, we tried to utilized min-batch k-means due to the fact it was computationally expedient in clustering, but due to the nature of k-means, this produced a consequent failure within the time domain. As seen below

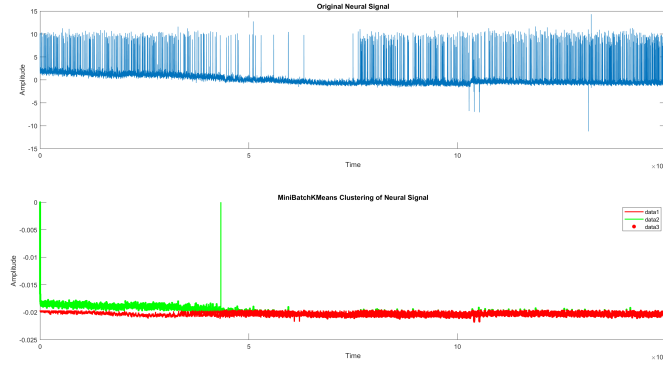


Figure 6: Result of min batch K means clustering on the SR filtered output

The k-means algorithm simply clusters the signal in half with the border simply being the mean of the signal. Perhaps better preprocessing or clustering the signal within the frequency domain would have produced better results, but due to the extremely poor performance, no more experiments were conducted using this method.

To train and predict using our SVM we must have a valid training set. This training set, as mentioned, is a Gaussian white noise profile that has been scaled 3 to match the amplitude of the input signal. After utilizing our selected SR parameters we then have the signal with the scaled 3 noise profile we have

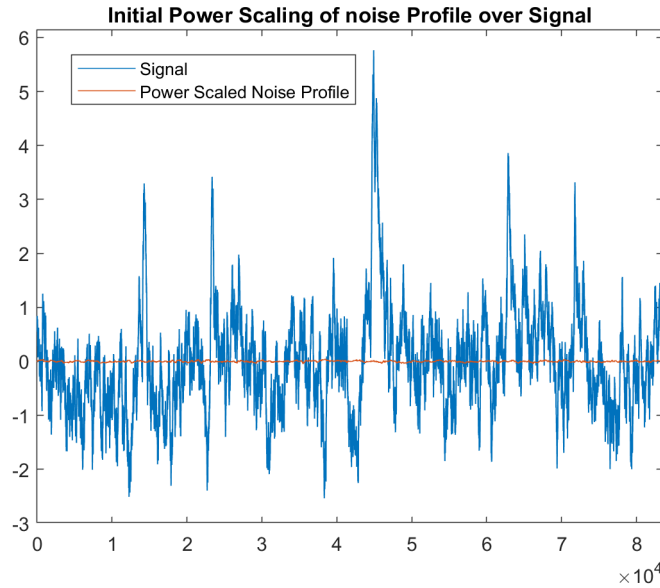


Figure 7: Power Scaled Noise on top of Filtered Noisy Signal

Visually, we can see that the noise profile scaled produces a non-representative noise profile for the filtered signal, thus scaling this even more produces

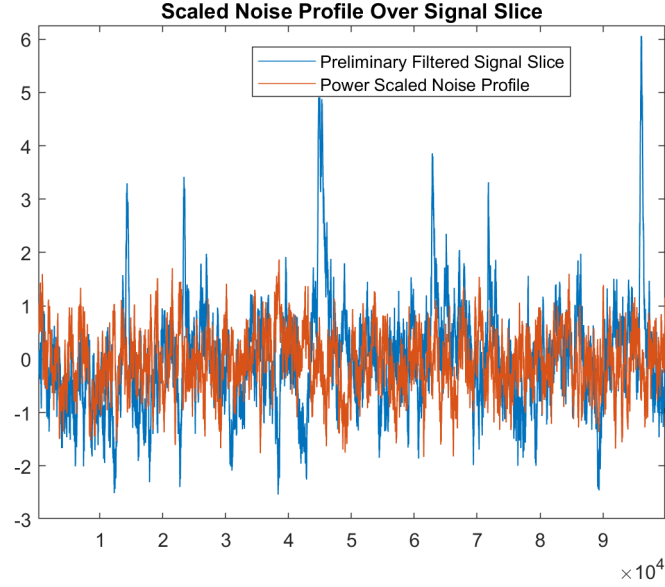


Figure 8: Correctly Power Scaled Noise Profile overlay with filtered signal output

which shows a much more representative training profile for the signals noise.

Due to the size limitations in MATLAB, the noise profile is limited to 1000 data points so the first 1000 data points are then utilized as the noise profile. The training of this SVM was exceptionally fast due to the non-complex nature of the noise profile. After training the SVM with the given noise profile⁸ we are then given a vector with the value of our SVs. Thus our predictive model must operate in batches to complete its task. During its predictive portion, the algorithm produces kernel values and decision values. Our classifying method is realized via simple thresholding. The threshold for any decision value is set at 0. This means any value less than zero (in the decision value matrix) indicates a spike portion. Utilizing our previously mentioned threshold of zero we can then cross reference the indices at which the decision values are less than zero, and label them within the signal slice as a spike. Values that show a high positive value, namely any decision value greater than 0, are labeled as noise portions. The post processing on this method involves negating negative values of the signal as the inherent voltage of the spikes is positive. Thus, any noise label that has a value greater than zero is also ignored due to this relationship. Compiling all of the above produces the following visual

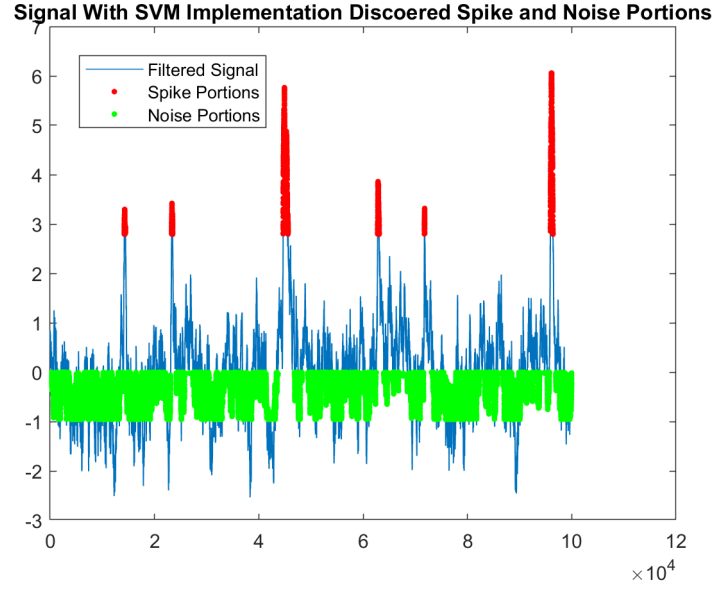


Figure 9: One Class SVM output on Filtered Signal with 8 as noise profile

As one can see while visually comparing the raw signal with its filtered, noisy output

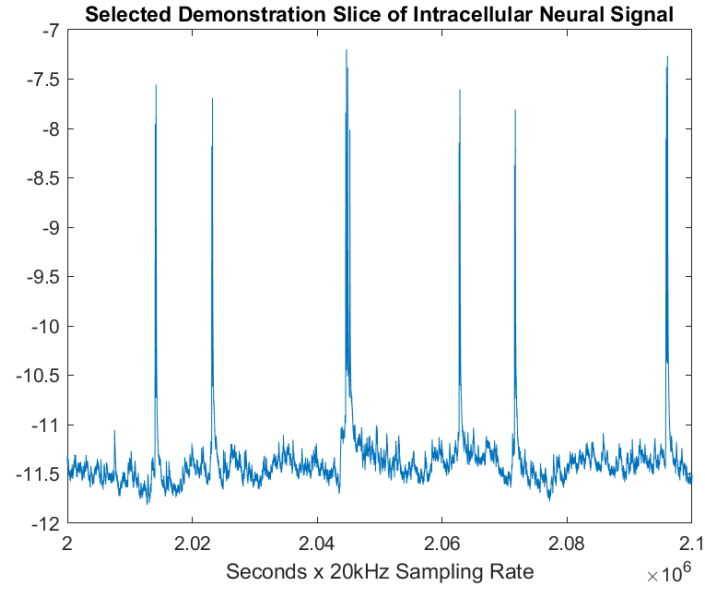


Figure 10: Raw signal slice

shows a clear success in terms of defining noise portions with high confidence. Even if there were portions of the signal that are not calculated in the iterative optimization after this step, it would

not matter as we would ignore them due to the post processing of the SVM output.

4 Conclusions

Upon review of the results it is clear that the desired goal has been achieved. That is, finding a metric in which we can filter a neural signal without ground truth data, optimize that metric to find a preliminary filtering of the signal for use preliminary classification, and classify points of high confidence in the signal. The particle swarm can efficiently find solutions to the SR parameters that produces a desirable output, even in preliminary classification. Which lends itself to its reusability for later additions within the script. The reusability comes in the form of calculating the SNR based on the preliminary classifiers of the signal, and re-optimizing the potential well solution to produce an even better signal resolution. Ideally this algorithm would only have to operate once every ten minutes to account for potential additions of noise within the signal measuring.

Given the following results, it should be tested that utilizing the proposed method in the discussion provides a better SR resonance output than a non-blind maximization method. This result is especially promising due to the fact that the actual use of this algorithm will be in optimizing solutions for noisy signals where the spike locations are unknown, which provides an already better solution than the optimization utilizing the SNR search space alone.

Further implementations of this method need to be carried out to verify its efficacy on extracellular signals, which is where the primary point of interest in the field of study lies. Utilizing the above methods in conjunction with one another provides a robust, iterative method, that with a small initialization delay, can produce fast and reliable neural signal filtering performance for more advanced neuro computational research work.

Project Work Distribution

M. Antonopoulos 40% Problem formulation, scripting methodology, data collection and analysis

N. Marosok - 50% experiemental procedure, conducted experiments via scripting on lab machines, scripted all of the experiments within the project. Only I could conduct experiments due to the hardware limitations both of us faced. I had access to extremely powerful lab machines that could compute through the scripts relatively fast whereas Mathieu did not have access to any such machine. Thus his role was mainly to create scripts and devise new ways to solve bugs within the current code or what the direction of the code should be.

References

- [1] Cihan Berk Güngör et al. “Investigating well potential parameters on neural spike enhancement in a stochastic-resonance pre-emphasis algorithm”. In: *J. Neural Engineering* 18.046062 (2021). DOI: <https://doi.org/10.1088/1741-2552/abfd0f>.
- [2] Cihan Berk Güngör and Hakan Toreyin. “Facilitating stochastic resonance as a pre-emphasis method for neural spike detection”. In: *J. Neural Engineering* 17.046047 (2020). DOI: <https://doi.org/10.1088/1741-2552/abae8a>.